Verification as a Control Signal: Integrity-Aware Scheduling for Resilient Parallel File Transfer

Given Name Surname
dept. name of organization
name of organization
City, Country
email address or ORCID

Given Name Surname
dept. name of organization
name of organization
City, Country
email address or ORCID

Given Name Surname
dept. name of organization
name of organization
City, Country
email address or ORCID

Given Name Surname
dept. name of organization
name of organization
City, Country
email address or ORCID

Given Name Surname
dept. name of organization
name of organization
City, Country
email address or ORCID

Given Name Surname
dept. name of organization
name of organization
City, Country
email address or ORCID

***Abstract*—Parallel chunked download is the standard way to move large files over wide-area networks, and every widely used implementation treats cryptographic verification as a terminal gate: bytes are scheduled on throughput and latency alone, checked once they arrive, and discarded if they fail. This paper argues that verification should instead be treated as a control signal, and reports what happens when it is. We present ReliaDL, a parallel transfer engine in which verification outcomes feed into both of the scheduler's decisions, which mirror to request a range from and how large that range should be. Sub-block Merkle verification aborts a corrupt transfer at the first bad 64 KB block and repairs only the unverified remainder rather than re-fetching the chunk. A concurrency constraint on the chunk-size controller prevents it from starving the worker pool, a failure mode the classical square-root sizing model does not predict because that model describes a single sequential stream. For mirror selection we report a result that corrected our own initial design: scoring a verification failure as a zero reward is necessary but not sufficient, because a fast mirror that corrupts a fraction of what it serves still has the highest expected reward and an adversarial bandit is therefore drawn towards it. We give the condition under which this occurs and show that displacement requires a mechanism reacting to failure rate rather than to average reward, which is an argument for why the cruder peer-banning used in peer-to-peer systems is the right shape. We evaluate on a deterministic fault-injecting testbed over 115 seeded runs across seven scenarios. The central result is that verification granularity determines whether a transfer completes at all, not merely how fast it is: under silent corruption, whole-file verification completes none of five runs while sub-block verification completes all five. The combined integrity path reduces wasted bytes from 44.7 to 4.2 percent of the payload and the share of bytes drawn from a corrupting mirror from 100 to 23.4 percent. The concurrency constraint improves throughput by between 4 and 145 percent. We report where the approach does not help, and release the engine, the harness and the raw data.**

***Index Terms*—parallel file transfer, data integrity, Merkle tree, multi-armed bandit, chunk scheduling, silent data corruption, fault tolerance.**

I. INTRODUCTION

Moving large files reliably across a wide-area network remains a routine and surprisingly unsolved problem. Scientific datasets, container images, machine-learning checkpoints and software distributions are all transferred in bulk by tools that split a file into byte ranges and fetch those ranges concurrently over HTTP. Range requests are standardised [1], parallel download accelerators are mature, and the engineering is widely considered settled.

What is not settled is how such a system should behave when the bytes that arrive are wrong. Transport-level corruption is handled by TCP checksums, but corruption that reaches the application intact-looking is not: a faulty middlebox, a misconfigured caching proxy, a failing disk on an origin server, or a mirror under an adversary's control can all deliver a well-formed response of exactly the right length whose contents differ from the original. End-to-end cryptographic verification is the only defence, and every parallel downloader we are aware of applies it the same way, as a terminal pass-or-fail gate. Bytes are scheduled using throughput and latency; verification happens afterwards; a failure discards work and triggers a retry.

This separation is costly in two ways, and the second is the more serious. First, discarding a whole chunk because of a single bad byte wastes bandwidth proportional to the chunk size. Second, a scheduler that never looks at verification outcomes cannot distinguish a mirror that is slow from a mirror that is wrong. A throughput-driven scheduler confronted with a fast mirror that corrupts a fraction of what it serves will prefer that mirror, because by the only signal it measures the mirror is excellent.

We therefore treat verification outcomes as a first-class input to the scheduler. Our starting intuition was that an adversarial bandit formulation already has the right shape: the EXP3 family [2] assumes nothing about how rewards are generated, admitting even an adversary choosing them to spite

the learner, and for mirror selection under possible corruption that assumption is simply accurate. We expected that scoring a failed verification as a zero reward would be enough to displace a corrupting mirror.

It is not, and establishing why is one of this paper's contributions. A mirror that is twice as fast and corrupts 40 percent of what it serves still returns more expected reward per request than an honest slower mirror, so a reward-maximising router is drawn towards it rather than away. We derive the condition under which this happens, confirm it against the shipped implementation, and show that displacement requires a mechanism that reacts to the failure rate rather than to the average reward.

This paper makes five contributions.

1) We formulate parallel chunked transfer as a problem with two coupled control decisions, mirror selection and chunk sizing, and show that both should read verification outcomes. We implement this in ReliaDL, a transfer engine of roughly 15,000 lines.

2) We show analytically and empirically that an integrity-aware reward alone does not displace a fast corrupting mirror, give the threshold at which a reward-maximising router is misled, and show that a consecutive-failure mechanism is what closes the gap.

3) We implement sub-block Merkle verification on the download path, so a corrupt chunk is repaired rather than re-fetched, and characterise the bound this achieves honestly: the saving depends on where in the chunk the corruption falls, and is zero in the worst case.

4) We show that the classical square-root chunk-sizing rule is wrong for a parallel downloader unless constrained by the worker count. The rule describes a single sequential stream and contains no term for concurrency; applied unmodified it grows chunks until fewer remain than there are workers, and measured throughput falls by up to 58 percent.

5) We evaluate on a deterministic fault-injecting testbed across 115 seeded runs and report negative results alongside positive ones. Adaptive chunk sizing reaches only parity with a well-chosen fixed size in our testbed, mirror routing is substantially worse than not routing when the origin is already the best available mirror, and aria2 is roughly 1.6 times faster than our engine on an unimpaired path.

The engine, the measurement harness, the scenario definitions and the raw result data are released so that every number here can be regenerated from a seed.

II. RELATED WORK

A. Parallel and multi-source transfer

Splitting a transfer across concurrent connections is long-established practice. Multi-source HTTP proposals [3] address scheduling across connections of differing quality, and multi-path streaming systems such as MSPlayer [4] compare chunk schedulers against proportional baselines, though in a video-streaming context where deadline pressure dominates. Production accelerators such as aria2 implement multi-source download with per-chunk checksums where a metalink provides them. In all of these, scheduling and verification remain separate concerns: the scheduler optimises for time-to-completion and the verifier reports a verdict afterwards.

B. Chunk and request sizing

Request granularity has received less attention than connection count. Recent work on short-video preloading observes directly that the effect of HTTP range-request granularity on the amount of data per request is underexplored [5]. Where sizing has been formalised it has generally been in streaming settings: WiFlix [6] applies Lyapunov drift-plus-penalty optimisation [7] to select video quality and requested chunk volume under a queue-stability constraint, and related work applies Lyapunov indexing to wireless file downloading under a power constraint [8]. These target deadline-driven streaming rather than bulk transfer, and the control action is coupled to a playout buffer that bulk download does not have. The sizing rule we analyse is the elementary square-root tradeoff between a fixed per-request cost and a cost linear in request size, the same first-order argument that yields the economic order quantity; our contribution is not the rule but the observation that it omits concurrency.

C. Integrity verification during transfer

Merkle trees [9] are the standard construction for authenticating parts of a large object, and are used in BitTorrent v2 and in content-addressed stores. FIVER [10] is the closest prior work to our integrity layer: it computes checksums at fixed chunk boundaries during a high-speed transfer and overlaps verification with transfer to reduce overhead, reporting a reduction from roughly 60 percent to under 10 percent. FIVER's concern is the cost of verification; ours is what the verdict should be used for once obtained. Incremental and homomorphic hashing [11], [12] allows a whole-object digest to be updated from part digests, which is attractive for the aggregation step but does not by itself localise a fault. We note that implementations duplicating odd nodes must bind the leaf count into the root or inherit a known second-preimage weakness [13]; we return to this in Section IV-F.

D. Bandits under corruption

The adversarial bandit setting and the EXP3 and EXP3.S algorithms are due to Auer et al. [2]. A separate line of work studies stochastic bandits robust to adversarial corruption of the reward stream [14], [15], with regret guarantees degrading gracefully in a corruption budget. That literature corrupts the reward signal; our setting corrupts the payload, and the reward is an honest report of a payload that was wrong. The distinction matters for the result in Section IV-D: because the reward is honest, a corrupting arm is not an anomaly to be filtered but a genuinely high-reward arm, and robustness to reward corruption does not address it. To our knowledge the application of a bandit to mirror selection with verification outcomes in the reward has not been studied, and restless-bandit formulations sometimes proposed for source selection [16], [17] do not address integrity at all.

E. Peer banning in BitTorrent

One body of practice does already connect verification to source selection, and it would be wrong to claim otherwise. BitTorrent clients verify each piece against its hash and commonly penalise or ban peers that repeatedly supply pieces which fail, so a peer serving bad data loses the right to serve. That is verification driving source choice, and it predates this work by two decades.

Two things distinguish our setting. The first is that BitTorrent's mechanism is a ban triggered by a count of failures, not an adjustment to a utility estimate; it is, in the terms of Section IV-D, a failure-rate mechanism rather than a reward-based one. Our contribution is not to propose such a mechanism but to explain why the reward-based alternative, which is what a modern scheduler would naturally reach for, does not work: we give the threshold (13) at which a reward-maximising router is actively drawn towards a corrupting source, and show the gap experimentally. Read this way, our Section VI-D result is an argument for why BitTorrent's cruder mechanism is the right shape.

The second difference is scope. Banning answers which source to avoid; it says nothing about how large a request should be, and BitTorrent's piece size is fixed at torrent creation. The chunk-sizing half of our formulation, and its interaction with detection latency through (7), has no analogue there.

F. Summary of the gap

Outside the peer-to-peer practice just described, verification is uniformly a post-hoc check. We found no prior work that treats a verification outcome as an input to request sizing at all, and none outside BitTorrent-style banning that uses it for source selection. Nor have we found an analysis of why the reward-based formulation fails, which Section IV-D supplies.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Transfer model

A client retrieves a file of size F bytes, available at M mirror URLs serving identical content. The file is partitioned into an ordered set of n chunks, chunk i covering byte range [s_i, e_i] of size S_i, with

$$\Sigma_{i=1..n} S_i = F, \quad s_{i+1} = e_i + 1 \qquad (1)$$

A pool of W workers issues HTTP range requests concurrently; each worker fetches one chunk at a time, writes received bytes to their absolute offset in a pre-allocated output file, and reports completion. Completed chunks are checkpointed so an interrupted transfer resumes at chunk granularity.

The scheduler makes two decisions: for each request it selects a mirror m from the M available, and it selects the chunk size S, which determines the partition. We treat these as the control variables.

B. Fault model and threat model

A mirror may exhibit four behaviours, independently and with mirror-specific rates. It may refuse a request, returning an error status. It may truncate a response, closing the connection after declaring a full content length. It may serve bytes slowly, at a per-connection rate below the client's capacity. Critically, it may serve silently corrupted bytes: a response of the correct length and status whose contents differ from the original in one or more positions. We write q for the fraction of a mirror's responses that are corrupted.

We assume an adversary who controls a strict subset of the mirrors and may return arbitrary bytes for any range request, including bytes crafted to appear plausible. The adversary observes which ranges are requested from the mirrors it controls and may vary its behaviour over time, for instance serving correctly until it has accumulated routing preference and then corrupting. It cannot forge the signed manifest, break SHA-256 preimage resistance, or modify traffic to honest mirrors. We assume the client possesses an authentic manifest binding the expected digests, obtained out of band; manifest distribution is orthogonal and well studied.

Under these assumptions the client cannot prevent a corrupt response, only detect it and respond. The quantity to minimise is therefore not the probability of receiving bad bytes, which the adversary controls, but the volume of traffic the adversary can induce before being displaced. This is what makes detection latency, rather than detection alone, the figure of merit.

C. Metrics

Let F be the delivered file size, T the wall-clock duration and B_m the bytes written by mirror m. We measure goodput

$$G = F / T \qquad (2)$$

and wasted bytes, the traffic that did not contribute to the delivered file,

$$\Delta = \Sigma_m B_m - F, \quad \delta = \Delta / F \qquad (3)$$

Δ is measured at the servers rather than inferred at the client, because a client that abandons a stream cannot observe the bytes it avoided. We also measure source attribution, the fraction of traffic drawn from mirrors known to corrupt,

$$\sigma = (\Sigma_{m \in bad} B_m) / (\Sigma_m B_m) \qquad (4)$$

which measures directly whether the scheduler is being misled, and completion rate, the fraction of attempts delivering a file matching its expected digest. In a setting with silent corruption the last of these is not a formality: a transfer that cannot repair a corrupt chunk does not complete slowly, it fails.

D. Why the decisions are coupled

The two control decisions interact through verification. A larger chunk amortises per-request overhead but increases the bytes at risk from a single corruption event, and reduces the number of verification events per byte, which delays detection of a corrupting mirror. A smaller chunk detects sooner and wastes less per failure but pays more overhead. Sub-block verification partially decouples these, because it makes the bytes lost to one corruption depend on the sub-block size rather than the chunk size. The relationship between detection latency and goodput is therefore a tradeoff surface rather than a single optimum, governed jointly by the chunk size and the verification granularity.

IV. DESIGN

A. Architecture

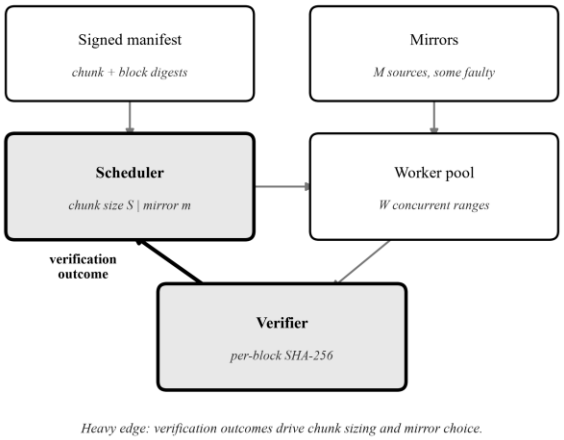


Fig. 1. Control flow in ReliaDL. The thin edges are present in any parallel downloader. The heavy edge, from the verifier back into the scheduler, is what this work adds: verification outcomes become an input to chunk sizing and mirror selection rather than only a terminal verdict.

Fig. 1 shows the arrangement. ReliaDL separates the transfer engine from the policies that drive it through a scheduling interface. The engine owns connection management, range requests, positional writes and checkpointing. The scheduler owns the two control decisions and the estimators that inform them. Each policy can be enabled independently, which is what makes an ablation possible: every configuration measured in Section VI differs from the control in exactly one respect.

B. Sub-block verification and localised repair

Each chunk is subdivided into blocks of size b, with b = 64 KB by default, and the manifest carries a SHA-256 digest per block in addition to the per-chunk digest. As a chunk streams in, each completed block is verified against its digest. A mismatch aborts the stream immediately, so the remainder of the chunk is never transferred.

The verified prefix is retained. Blocks before the failure matched their digests and their bytes are already durable on disk, so the retry requests only from the start of the failed block. For a corruption in block k of a chunk of size S, the bytes re-requested are

$$R(k) = S - k \cdot b, \quad k \in \{0, \ldots, \lceil S/b \rceil - 1\} \qquad (5)$$

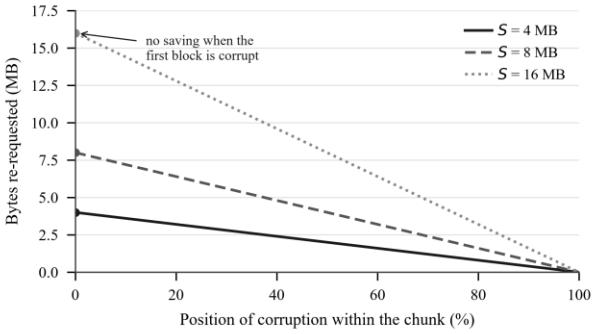


Fig. 2. Equation (5): bytes re-requested after a corruption, against where in the chunk it falls. The left-hand end is the point of the figure. Localisation saves nothing when the first block is corrupt, and that worst case costs exactly what no localisation costs.

Fig. 2 plots this. The saving is position-dependent, which is the honest statement of what the technique achieves and is weaker than it is usually described. Corruption in the final block saves almost the entire chunk; corruption in block zero saves nothing. Treating the position as uniform over the chunk gives

$$E[R] = S/2 + b/2 \approx S/2 \qquad (6)$$

so the expected repair is halved rather than reduced to a single block. A position-independent cost of one block per corruption is achievable, but not with early abort: it would require verifying the chunk to completion, collecting every bad block, and re-requesting exactly those ranges. That trades the bandwidth saved by aborting against a smaller repair, and delays the verdict on the mirror. Which side wins depends on q, S and b; we implement early abort and return to the alternative in Section VII.

C. Detection latency

The chunk size determines how quickly a corrupting mirror is discovered, because verification occurs once per chunk and so the number of verification events per byte is inversely proportional to S. If a mirror corrupts a fraction q of its responses, the number of its responses until the first failure is geometric with mean 1/q, so the expected volume it delivers before its first zero-scored request is

$$E[D] = S / q \quad \text{bytes} \qquad (7)$$

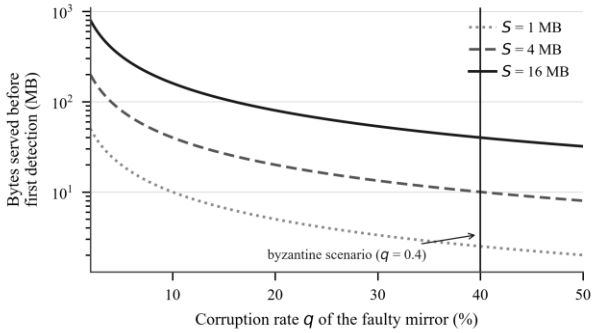


Fig. 3. Equation (7): bytes a corrupting mirror delivers before its first detected failure. Smaller chunks detect in fewer bytes, in direct opposition to the throughput argument for larger chunks. The marked rate is the one used in the byzantine scenario of Section VI.

Fig. 3 shows the relationship. Note that b does not appear in (7): localisation bounds the bytes wasted per failure, not the bytes delivered before the first failure is observed. The two parameters control different quantities, which is why both are needed.

D. Why an integrity-aware reward is not enough

Mirror selection is treated as an adversarial bandit over M arms. Throughput depends on other tenants' load, routing changes and cache state, none of which is stationary or observable, and only the reward of the mirror actually used is revealed. We use EXP3.S [2], whose selection distribution is

$$p_{m,t} = (1 - \eta)\cdot w_{m,t} / \Sigma_j w_{j,t} + \eta/M \qquad (8)$$

with an importance-weighted reward estimate and multiplicative update

$$\hat{r} = r / p_{m,t}, \quad w_{m,t+1} = w_{m,t}\cdot\exp(\eta\cdot\hat{r}/M) \qquad (9)$$

The uniform η/M floor serves two purposes. It keeps every mirror under occasional test so a recovered edge can be rediscovered, and it bounds every selection probability away from zero, so that

$$\hat{r} \le M/\eta, \quad \text{and therefore} \quad \eta\cdot\hat{r}/M \le 1 \qquad (10)$$

which is what makes the importance-weighted estimate finite and the per-round exponent bounded. Weights are maintained in logarithmic space: the bound in (10) is per round and accumulates, so a weight grows as e^T over T rounds and a long transfer issues tens of thousands of requests. Weights are rescaled after each update so the largest is unity, which leaves (8) unchanged because it depends only on weight ratios.

Classical EXP3 has vanishing regret against the best fixed arm, which is the wrong target here, since over a long transfer the best mirror changes. The EXP3.S decay mixes each weight toward the mean each round,

$$w_m \leftarrow (1 - \alpha)\cdot w_m + (\alpha/M)\cdot\Sigma_j w_j \qquad (11)$$

bounding how much evidence any arm can accumulate and converting an algorithm that converges into one that tracks. We set $\alpha \approx 1/T$ following the standard guidance, confirmed empirically to minimise worst-case regret at horizons T = 1000, 2000, 5000 and 10000 under stationary, abruptly switching and gradually drifting reward schedules.

The integrity coupling we first implemented was minimal. The reward is the achieved goodput of a request, normalised to the pool's observed peak and clipped to the unit interval, and a request that fails scores exactly zero; a chunk failing verification is a request that failed. We expected this to suffice. It does not, and the reason is worth stating precisely.

Let an honest mirror deliver normalised reward r_h on every request, and let a faster mirror deliver r_f > r_h but corrupt a fraction q of its responses, scoring zero on those. Their expected rewards are

$$E[r_{bad}] = (1 - q)\cdot r_f, \quad E[r_{good}] = r_h \qquad (12)$$

so a reward-maximising router prefers the corrupting mirror precisely when

$$q < 1 - r_h / r_f \qquad (13)$$

With a mirror twice as fast as its honest peers, (13) gives q < 0.5: a mirror may corrupt up to half of everything it serves and still be the bandit's favourite. This is not a pathological case but the common one, because a mirror that is fast is attractive for the same reason whether or not its bytes are correct, and the reward signal cannot separate the two.

The consequence is that routing on reward alone, however integrity-aware that reward is, does not solve the problem. What is needed is a mechanism responding to the failure rate rather than to the average. We use a consecutive-failure breaker: after c successive failures a mirror is removed from the candidate set for a cooldown that grows geometrically with repeated trips. For a mirror with corruption rate q, the probability that any given window of c consecutive requests are all failures is q^c, so the expected number of requests before the breaker trips is

$$E[N] = (1 - q^c) / ((1 - q)\cdot q^c) \qquad (14)$$

which for q = 0.4 and c = 3 is about 25 requests, after which the mirror is excluded for the cooldown period regardless of how attractive its average reward remains. Section VI-D quantifies the difference the two mechanisms make.

E. Concurrency-constrained chunk sizing

The chunk-size controller minimises a per-byte cost trading per-request overhead against bytes lost to a failed request:

$$J(S) = \tau\cdot\mu/S + p_{seg}\cdot S/MSS \qquad (15)$$

where τ is per-request overhead in seconds, μ is estimated throughput, p_seg the per-segment drop probability and MSS the maximum segment size. Setting

$$dJ/dS = -\tau\cdot\mu/S^2 + p_{seg}/MSS = 0 \qquad (16)$$

gives the square-root optimum, to which a bandwidth-delay term is added so a fast clean path is not throttled by request overhead:

$$S^* = \gamma\cdot\sqrt{(\tau\cdot\mu\cdot MSS / p_{seg})} + \lambda\cdot\mu\cdot RTT \qquad (17)$$

The estimators observe failures per chunk while the loss model is expressed per segment; at an 8 MB chunk the two differ by roughly four orders of magnitude. The measured chunk failure ratio is therefore converted using the transfer size that produced the observation,

$$p_{seg} = 1 - (1 - p_{chunk})^{(MSS / S_{obs})} \qquad (18)$$

without which the controller is dimensionally incoherent in practice even though (17) is correct on paper.

Equation (15) describes a single sequential stream. It contains no term for concurrency, and that omission dominates. Writing r for the per-connection rate and C for any shared capacity, aggregate throughput is approximately

$$T = \min(n_{flight}\cdot r , C), \quad n_{flight} = \min(W, F_{rem}/S) \qquad (19)$$

so once S exceeds F_rem/W the number of chunks in flight falls below W and workers idle. The parallelism lost then costs far more than the request overhead saved. The controller walks directly into this: on a clean path it observes no loss, drives S toward its ceiling by (17), and starves the pool. We therefore constrain the solution,

$$S = \min(S^*, \max(S_{min}, F_{rem}/W)) \qquad (20)$$

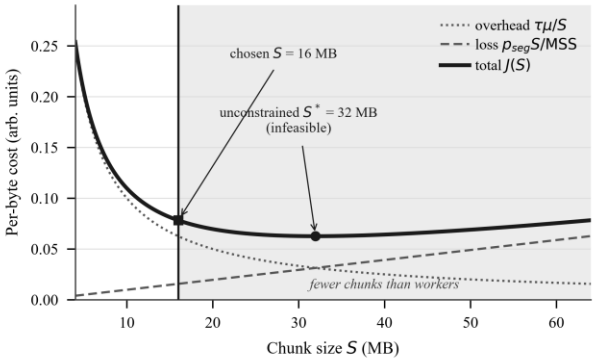


Fig. 4. Equations (15), (17) and (20) on a clean path. The cost minimum at 32 MB lies inside the region where fewer chunks remain than there are workers, so the unconstrained optimum is infeasible and the constraint selects 16 MB. The cost model alone cannot see the shaded region, because it contains no term for concurrency.

Fig. 4 shows why the constraint is needed rather than merely prudent: the unconstrained minimum of (15) lies inside the infeasible region. The constraint only ever shrinks a chunk, so the loss-driven lower bound computed by (17) is untouched; it restricts the feasible set rather than altering the objective. Chunks are issued lazily during the transfer rather than planned in advance, so the controller re-reads the network state as conditions change, and a chunk shortened by (20) returns its unused bytes to the planner so that (1) continues to hold.

F. Binding the leaf count into the Merkle root

Our tree uses domain separation, hashing leaves and internal nodes as

$$H_{leaf}(d) = H(0x00 \,\|\, d), \quad H_{node}(L,R) = H(0x01 \,\|\, L \,\|\, R) \qquad (21)$$

and pairs an odd trailing node with itself. That last step is the standard construction and carries a known weakness [13]: a chunk list whose tail is duplicated produces identical internal nodes and therefore the same root as the original list. Since the root is signed, two different chunk sequences would share one signature. We close this by committing to the leaf count,

$$root = H(0x02 \,\|\, uint64_be(n) \,\|\, tree_root) \qquad (22)$$

which leaves the tree shape, and therefore every existing inclusion proof, unchanged. We verified the collision experimentally before the change and its absence afterwards.

V. IMPLEMENTATION

ReliaDL is implemented in approximately 15,000 lines of Python, using asyncio and a single connection pool. Blocking file operations are offloaded to a thread pool rather than performed on the event loop; an earlier revision performed an fsync and a full state-file rewrite after every chunk in every worker, which serialised the entire transfer behind disk I/O and cost roughly 40 percent of wall-clock time on a local 96 MB transfer. Checkpointing is now batched, at a cost bounded by the re-download of at most the batch size after a crash. No chunk is recorded as complete before its bytes are durable, so the resume state never over-claims.

Received bytes are coalesced into a buffer before being written positionally, which reduces thread-pool handoffs from one per network block to one per buffer. A subtlety arises in combination with early abort: on a verification failure the buffer must be flushed before the stream unwinds, or the verified prefix of (5) is only partly on disk and the localised retry resumes from an offset whose bytes were never written. We encountered exactly this, and the symptom was that localisation measured slower than no localisation at all.

The system is covered by 1,007 automated tests, including property-level invariants for (1) under the constraint (20) and regression tests for the second-preimage case closed by (22).

VI. EVALUATION

A. Methodology

We evaluate against a configurable fault-injecting HTTP origin rather than a network emulator such as netem. The decisive reason is that netem cannot produce the fault that matters here: it corrupts frames, which TCP's checksum catches and retransmits, whereas we need corruption that arrives looking intact. A programmable origin also measures what it impairs, so Δ in (3) is observed rather than inferred.

Impairments are drawn deterministically from a keyed hash of the seed, the requested byte range and the attempt number, rather than from a sequential pseudo-random stream. This matters for fairness: impairments then depend on which bytes are requested and not on the order in which requests happen to arrive, so two different schedulers fetching the same file encounter the same faults. Without this property an ablation compares two different fault sequences and attributes the difference to the policy. We verified it directly: repeating the matrix under identical seeds reproduces identical byte counts, and reordering cells does not change results.

Each measurement is a (scenario, policy, seed) triple. The payload is 64 MB, W = 4, and each cell is repeated over five seeds. We report medians with interquartile ranges, because transfer times are right-skewed: a single retry storm produces an outlier several times the typical run, and a mean with a standard deviation over five points describes that distribution poorly. With five repetitions these are descriptive statistics and we do not claim statistical significance.

Table I lists the scenarios and Table II the policies. Each policy differs from the control in exactly one respect.

TABLE I. EVALUATION SCENARIOS

Scenario	Mirrors	Characteristic
clean	1	No impairment; establishes the ceiling
lossy	1	15% of range requests fail
corrupting	1	q = 0.20, single source
heterogeneous	3	40, 20 and 8 MB/s per connection

Scenario	Mirrors	Characteristic
failover	2	Fastest collapses to 2 MB/s mid-transfer
byzantine	3	Fastest has q = 0.40
flaky	2	Failure, corruption and truncation

TABLE II. POLICY CONFIGURATIONS

Policy	Sizing	Routing	Sub-block
baseline	fixed	origin	no
adaptive-chunks	controller	origin	no
adaptive-mirrors	fixed	EXP3.S	no
localization	fixed	origin	yes
routing+localization	fixed	EXP3.S	yes

B. Verification granularity determines completion

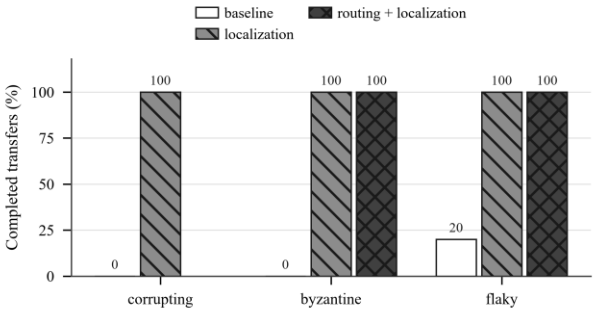


Fig. 5. Completion rate under silent corruption, five attempts per cell. Whole-chunk verification completes none of them on the corrupting and byzantine scenarios. Routing does not apply to the single-mirror corrupting scenario and is left blank there.

Fig. 5 and Table III report completion rates. This is the central result of the paper and it is not about speed. Under silent corruption, a configuration verifying only whole chunks against a whole-file digest completes none of five attempts. The corruption is detected, but only after the entire file has been transferred, by which point it cannot be attributed to any chunk and there is no recovery path short of restarting. With per-block digests the corrupt region is identified while the stream is still open, repaired, and every attempt completes.

TABLE III. COMPLETION RATE (SUCCESSES OUT OF FIVE)

Scenario	baseline	localization	routing + loc.
corrupting	0 / 5	5 / 5	—
byzantine	0 / 5	5 / 5	5 / 5
flaky	1 / 5	5 / 5	5 / 5

We emphasise this because the parallel-download literature is framed almost entirely around throughput. In a fault model that includes silent corruption, verification granularity is a liveness property.

C. Integrity-aware routing reduces waste and misdirection

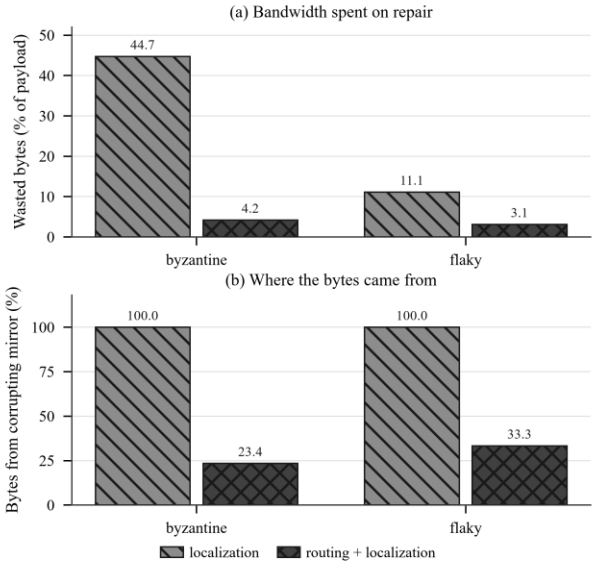


Fig. 6. Effect of routing on (a) bandwidth spent on repair, δ from equation (3), and (b) source attribution σ from equation (4). Two quantities with different units, so two panels sharing a policy axis rather than two scales on one plot.

Fig. 6 and Table IV report the byzantine scenario, in which the fastest of three mirrors has q = 0.40. Localisation alone repairs the corruption but keeps requesting from the corrupting mirror, because without routing there is nowhere else to go; it wastes 44.7 percent of the payload in repairs and draws every byte from the bad source. Adding routing reduces waste roughly tenfold and cuts the corrupting mirror's share to 23.4 percent, for a cost of roughly 22 percent of goodput.

TABLE IV. BYZANTINE SCENARIO: THREE MIRRORS, FASTEST HAS Q = 0.40

Policy	G (MB/s)	δ (%)	σ (%)
localization	38.2	44.7	100.0
routing + localization	29.9	4.2	23.4

D. Reward alone does not displace a corrupting mirror

Section IV-D predicts that a reward-maximising router is drawn towards a fast corrupting mirror whenever (13) holds. We test this directly by running the shipped router under both configurations against three mirrors, one of which is twice as fast and has q = 0.40. Condition (13) gives the threshold q < 0.5, so the corrupting mirror should be preferred.

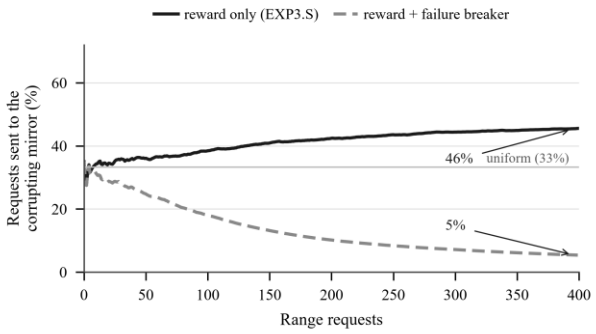


Fig. 7. Share of requests sent to a corrupting mirror that is twice as fast as its honest peers, averaged over 40 seeded trials of the shipped implementation. Routing on an integrity-aware reward alone sends it more traffic than uniform, as equation (13) predicts. The consecutive-failure breaker of equation (14) is what displaces it.

Fig. 7 confirms the prediction and is the figure that corrected our own design. With an integrity-aware reward and nothing else, the corrupting mirror ends with 45.5 percent of requests, which is not merely a failure to demote it but an active preference: uniform allocation over three mirrors would give it 33 percent. Adding the consecutive-failure breaker brings it to 5.4 percent.

The lesson generalises beyond this system. An average-reward learner is the wrong instrument for an integrity fault, because integrity failures are not noise in the reward but a property the reward deliberately averages away. The mechanism must react to the rate of failure, not to its mean effect on utility. We would not have found this by reasoning about the algorithm, and we report it because the opposite claim is the intuitive one and we initially made it.

E. Routing is not free

Table V shows both directions. When the origin degrades mid-transfer, exploration is what allows the router to discover the alternative, and routing improves goodput by 91 percent. When the origin is already the fastest of three mirrors, exploration is pure cost and routing loses 39 percent. The second scenario is constructed in the baseline's favour, since the client is pointed at the best available mirror, and we retain it precisely because it bounds the claim.

TABLE V. MIRROR ROUTING, BOTH DIRECTIONS (MB/S)

Scenario	baseline	adaptive-mirrors	Change
failover	8.6	16.5	+91%
heterogeneous	46.7	28.4	−39%

F. The concurrency constraint

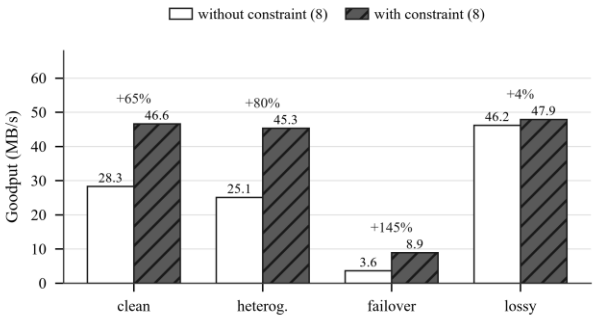


Fig. 8. Measured effect of constraint (20), with the two arms interleaved within one process so that drift in machine load falls on both equally. The failover case is the largest: a single oversized chunk landing on the degraded mirror stalls everything queued behind it.

Fig. 8 and Table VI report a controlled comparison of the chunk-size controller with and without (20). Unconstrained, the controller is roughly 40 percent slower than a fixed 4 MB chunk on a clean path. On failover it is worse still, at 3.6 MB/s against 8.9.

TABLE VI. CHUNK SIZING WITH AND WITHOUT CONSTRAINT (20) (MB/S)

Scenario	Unconstrained	Constrained	Change
clean	28.3	46.6	+65%
heterogeneous	25.1	45.3	+80%
failover	3.6	8.9	+145%
lossy	46.2	47.9	+4%

We state the limit of this result plainly. The constraint repairs a large regression; it does not by itself make adaptive sizing a win. Against a fixed 4 MB chunk the constrained controller lands between −0.5 and +5.8 percent across the four scenarios: 47.9 against 48.2 MB/s on clean, 48.9 against 48.2 on lossy, 49.4 against 46.7 on heterogeneous, and 8.9 against 8.6 on failover. Several of those comparisons sit inside overlapping interquartile ranges over five seeds, so what the data supports is that the controller costs nothing here, not that it beats a constant. An earlier version of this measurement found it 9 to 10 percent worse on lossy and heterogeneous; that was taken against an implementation whose loss term carried a stray bits-to-bytes factor, which made every loss-driven chunk a factor of sqrt(8) too small. The controller would still have to earn its place on paths where a good fixed size is not known in advance, and a single file size with a single worker count cannot demonstrate that.

G. Localised repair and engineering cost

For a single flipped byte in block 40 of a 64-block, 4 MB chunk, whole-chunk verification re-requests 4096 KB and sub-block localisation re-requests 1536 KB, matching (5) exactly. As Fig. 2 predicts, corruption placed in block zero saves nothing.

Verification is not free. The cryptographic cost is per-byte hashing, which is not the binding constraint here: the localisation policy reaches 45.1 MB/s on the corrupting scenario while verifying every block, against a 48.2 MB/s ceiling with no verification and no faults. The structural cost is larger and less obvious, as Section V describes.

H. Comparison with existing tools

On the clean scenario with four workers, ReliaDL achieves 50.7 MB/s against 23.9 MB/s for single-stream curl, and against 79.8 MB/s for aria2 configured with an explicit split and minimum split size. aria2 is therefore roughly 1.6 times faster than our engine on an unimpaired path, which is the expected shape for a mature C++ implementation against Python asyncio, and we make no claim to the contrary. The ordering reverses under corruption. Against a mirror silently corrupting one byte in twenty percent of responses, aria2 exited successfully on all five runs and delivered a full-size file failing its SHA-256 on two of them, with nothing in its output distinguishing those runs; our engine without a manifest refuses delivery, and with one it repairs and completes all five. The cost of being 1.6 times slower buys the difference between a wrong file reported as correct and a correct file.

I. Full results matrix

Table VII gives every cell of the matrix, including those omitted from the discussion above. Cells marked “n/a” were not run because the policy cannot differ from the control there: mirror routing needs more than one mirror, and sub-block digests change nothing where no corruption is injected. Cells marked “fail” completed no run; where completion was partial the count is given.

TABLE VII. MEDIAN GOODPUT (MB/S) ACROSS THE FULL MATRIX

Scenario	base	chunks	mirrors	local.	rout+loc
clean	48.2	47.9	n/a	n/a	n/a
lossy	48.2	48.9	n/a	n/a	n/a
corrupting	fail	fail	n/a	45.1	n/a
heterogeneous	46.7	49.4	28.4	n/a	n/a
failover	8.6	8.9	16.5	n/a	n/a
byzantine	fail	fail	fail	38.2	29.9
flaky	32.7 (1/5)	36.3 (1/5)	25.6 (3/5)	32.3	25.5

Two patterns are visible across the whole matrix that the selected tables do not show. First, the policies that help are concentrated entirely in the lower three rows, the scenarios with faults; on clean and lossy paths every policy is at best neutral and the control is the right choice. Second, partial completion is itself informative: on the flaky scenario the three policies without sub-block digests complete between one and three runs of five, so their goodput figures are medians over the subset that happened to succeed and are not comparable with the integrity policies, which completed every run. We give the counts rather than the medians alone for that reason.

VII. DISCUSSION AND LIMITATIONS

Several limitations bound what these results support, and we prefer to state them than to let a reader discover them.

Our testbed is loopback on a single machine. There is no meaningful propagation delay, so τ in (15) is understated relative to a wide-area transfer and the bandwidth-delay term of (17) is small. Bandwidth is modelled per connection rather than as a shared bottleneck, which in the notation of (19) means C is effectively unbounded and T is set by $n_flight \cdot r$. This is the common case for real origins, and precisely why parallel downloaders are effective, but it is also the condition most favourable to parallelism; where C binds, additional connections divide one capacity and buy only latency hiding. Every speedup we report is a speedup against a per-connection limit.

Our impairments are emulated rather than produced by a network stack. The waste and completion results are byte counts and are unaffected by the fidelity of the timing model, but the goodput figures are not, and a comparison against kernel traffic shaping remains outstanding.

We use five repetitions, one file size and one worker count. The chunk-sizing result in particular is a claim about a tradeoff, and a tradeoff is properly characterised by a curve rather than a point.

The early-abort decision deserves re-examination in light of (6) and (7). Aborting saves the remainder of the current chunk but leaves the repair cost position-dependent. Verifying through to the end of the chunk would make the repair cost proportional to the number of corrupt blocks rather than to their position, at the price of transferring the whole chunk and delaying the verdict of (7). Under a high corruption rate the second is likely to dominate, since most chunks fail anyway and the bytes saved by aborting are small relative to the repeated repairs; under a low rate the first should win. We have not measured the crossover, and it is the single experiment we would run next.

Equation (14) assumes a fixed corruption rate. An adversary free to vary q over time, corrupting just rarely enough to avoid c consecutive failures, is not covered by it and would retain a larger share than Fig. 7 suggests. We regard this as the most important gap in the security argument.

Finally, a design constraint worth recording. Sub-block localisation requires a manifest, and a manifest's digests are computed over particular chunk boundaries, while adaptive sizing exists to move those boundaries during the transfer. The two are therefore mutually exclusive as formulated: when a manifest is supplied the engine must adopt its partition, and (20) does not apply. This is not an implementation shortcut. Any scheme retaining both must publish digests at a granularity independent of the chunking, which is in fact what the fixed-size sub-blocks of Section IV-B already are and is the natural route out.

A. Generality

Nothing in the design is specific to HTTP range requests. The three ingredients it needs are that content be retrievable in independently addressable pieces, that each piece be independently verifiable against an authenticated digest, and that more than one source be available. Several widely deployed systems satisfy all three. BitTorrent v2 publishes per-piece Merkle data and maintains a swarm of peers of varying trustworthiness, which is the byzantine scenario by construction rather than by configuration. Content-addressed stores verify every block by construction and typically have several providers. Object stores expose range reads and, with a signed manifest alongside, admit the same treatment.

In each case the scheduler already exists and already ignores the verification outcome it computes. The contribution of this paper is less a new mechanism than the observation that a signal these systems already produce is being discarded, together with the finding of Section VI-D that using it naively is not enough.

VIII. FUTURE WORK

Four directions follow directly from the limitations above, in the order we judge most useful.

First, characterise the frontier. Equation (7) ties detection latency to S and (5) ties repair cost to b, but we measure a single point on a two-dimensional surface. Sweeping S and b jointly, over several worker counts and file sizes, would turn the tradeoff we describe qualitatively into a curve a practitioner could use for parameter selection.

Second, compare early abort against verify-through, as discussed in Section VII, and locate the crossover in q.

Third, move the evaluation onto real paths. A wide-area transfer with a non-trivial bandwidth-delay product tests (17) where it should matter most, and a shared bottleneck link would establish how much of the measured parallel speedup survives when C in (19) binds.

Fourth, strengthen the routing guarantee. Section VI-D shows that an average-reward learner is drawn to a fast corrupting mirror and that a failure-rate mechanism corrects it, but we offer no bound on the bytes an adversary can extract before displacement. Equation (14) gives an expectation under a fixed q; an adaptive adversary staying just below the breaker threshold is not covered. Deriving a bound against such an adversary, perhaps by adapting the corruption-tolerant bandit results [14], [15] to payload rather than reward corruption, would convert an empirical observation into a guarantee and is the direction we consider most valuable.

IX. CONCLUSION

Parallel file transfer systems schedule on throughput and verify afterwards. We have argued that the verification outcome is itself a scheduling signal, and shown what follows from using it. Under a fault model including silent corruption, verification granularity determines whether a transfer completes at all rather than how quickly: whole-file verification completed none of five attempts where sub-block verification completed all five, and the combined integrity path reduced wasted bytes roughly tenfold while cutting a corrupting mirror's share of delivered traffic from 100 to 23.4 percent.

The result we did not expect is that scoring verification failures into the reward is necessary but not sufficient. A fast mirror corrupting 40 percent of what it serves remains the highest-reward arm, and an adversarial bandit routes towards it; displacement requires a mechanism that reacts to the failure rate. We give the threshold at which this occurs and confirm it against the implementation. Separately, we showed that the classical square-root chunk-sizing rule omits concurrency and degrades throughput by up to 58 percent in a parallel downloader unless constrained by the worker count.

We have also reported where the approach does not help: adaptive sizing only reaches parity with a well-chosen fixed chunk size in our testbed, routing is costly when the origin is already the best available mirror, and aria2 moves unimpaired bytes substantially faster than we do. We regard the first as the more instructive, because it suggests the value of an adaptive controller lies not in outperforming a good fixed parameter but in removing the need to know one in advance, and that is a claim our evaluation is not yet structured to test.

X. ARTIFACT AVAILABILITY

The transfer engine, the fault-injecting origin, the scenario and policy definitions, the measurement harness and the raw result data for all 115 runs are released under the Apache 2.0 licence. Every table and measured figure in this paper can be regenerated from a seed with a single command, and the harness records the host environment alongside the results, since absolute figures are not comparable across machines. Figures 4 and 7 are produced by the shipped implementation rather than a reimplementation. A script comparing the emulated origin against kernel traffic shaping is included for use on platforms where that is available.

ACKNOWLEDGEMENT

[Acknowledgements to be completed by the authors.]

REFERENCES

[1] R. Fielding, Y. Lafon, and J. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Range Requests,” RFC 7233, IETF, 2014.

[2] P. Auer, N. Cesa-Bianchi, Y. Freund, and R. E. Schapire, “The Nonstochastic Multiarmed Bandit Problem,” SIAM Journal on Computing, vol. 32, no. 1, pp. 48–77, 2002.

[3] J. Kim, Y. Kim, and K. Lee, “Multi-Source Multi-Path HTTP (mHTTP): A Proposal,” arXiv:1310.2748, 2013.

[4] Y.-S. Lim, Y.-C. Chen, E. M. Nahum, D. Towsley, and K.-W. Lee, “MSPlayer: Multi-Source and Multi-Path Leveraged YoutubER,” arXiv:1406.6772, 2014.

[5] “DeLoad: Demand-Driven Short-Video Preloading with Scalable Watch-Time Estimation,” arXiv:2510.18459, 2025.

[6] A. Destounis, M. Assaad, M. Debbah, and B. Sayadi, “WiFlix: Adaptive Video Streaming in Massive MU-MIMO Wireless Networks,” IEEE Transactions on Wireless Communications, vol. 15, no. 6, pp. 4088–4103, 2016.

[7] M. J. Neely, Stochastic Network Optimization with Application to Communication and Queueing Systems. Morgan and Claypool, 2010.

[8] X. Wei and M. J. Neely, “Power Aware Wireless File Downloading: A Lyapunov Indexing Approach to a Constrained Restless Bandit Problem,” arXiv:1504.04074, 2015.

[9] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in Advances in Cryptology — CRYPTO ’87, LNCS 293, pp. 369–378, 1988.

[10] E. Arslan and A. Alhussen, “Fast End-to-End Integrity Verification for High-Speed File Transfers,” arXiv:1811.01161, 2018.

[11] M. Bellare and D. Micciancio, “A New Paradigm for Collision-Free Hashing: Incrementality at Reduced Cost,” in Advances in Cryptology — EUROCRYPT ’97, LNCS 1233, pp. 163–192, 1997.

[12] K. Lewi, W. Kim, I. Maykov, and S. Weis, “Securing Update Propagation with Homomorphic Hashing,” IACR Cryptology ePrint Archive, Report 2019/227, 2019.

[13] “Merkle Tree Second-Preimage Vulnerability from Duplicated Trailing Nodes,” CVE-2012-2459, 2012.

[14] T. Lykouris, V. Mirrokni, and R. Paes Leme, “Stochastic Bandits Robust to Adversarial Corruptions,” in Proc. 50th ACM Symposium on Theory of Computing (STOC), pp. 114–122, 2018.

[15] S. Kapoor, K. K. Patel, and P. Kar, “Corruption-Tolerant Bandit Learning,” Machine Learning, vol. 108, no. 4, pp. 687–715, 2019.

[16] P. Whittle, “Restless Bandits: Activity Allocation in a Changing World,” Journal of Applied Probability, vol. 25A, pp. 287–298, 1988.

[17] R. R. Weber and G. Weiss, “On an Index Policy for Restless Bandits,” Journal of Applied Probability, vol. 27, no. 3, pp. 637–648, 1990.

[18] D. Chase and Y. Lev, “Dynamic Circular Work-Stealing Deque,” in Proc. 17th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA), pp. 21–28, 2005.

[19] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing TCP’s Retransmission Timer,” RFC 6298, IETF, 2011.

[20] A. Rundgren, B. Jordan, and S. Erdtman, “JSON Canonicalization Scheme (JCS),” RFC 8785, IETF, 2020.

[21] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.